forces us to formulate our problems in a way that is amenable to dynamic programming-like problem decomposition, such that we can iteratively optimize over local sub-problems and make progress toward achieving the global high-level objective. Fortunately, it's pretty simple too.

To concretely illustrate the framework, let's consider the task of building an RL algorithm that can learn to minimize the energy usage at a big data center. Computers need to be kept cool to function well, so large data centers can incur significant costs from cooling systems. The naive approach to keeping a data center cool would be to keep the air conditioning on all the time at a level that results in no servers ever running too hot; this would not require any fancy machine learning. But this is inefficient, and you could do better, since it's unlikely that all servers in the center are running hot at the same times and that the data center usage is always at the same level. If you targeted the cooling to where and when it mattered most, you could achieve the same result for less money.

Step one in the framework is to define your overall objective. In this case, our overall objective is to minimize money spent on cooling, with the constraint that no server in our center can surpass some threshold temperature. Although this appears to be two objectives, we can bundle them together into a new composite *objective function*. This function returns an error value that indicates how off-target we are at meeting the two objectives, given the current costs and the temperature data for the servers. The actual number that our objective function returns is not important; we just want to make it as low as possible. Hence, we need our RL algorithm to minimize this objective (error) function's return value with respect to some input data, which will definitely include the running costs and temperature data, but may also include other useful contextual information that can help the algorithm predict the data center usage.

The input data is generated by the *environment.* In general, the environment of a RL (or control) task is any dynamic process that produces data that is relevant to achieving our objective. Although we use "environment" as a technical term, it's not too far abstracted from its everyday usage. As an instance of a very advanced RL algorithm yourself, you are always in some environment, and your eyes and ears are constantly consuming information